Intelligent Placement Assistant: A Retrieval-Augmented Generation (RAG) System for Campus Recruitment Support

Oshish Madhan
Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology
Email: omadhan_be24@thapar.edu

August 13, 2026

Project Proposal for UCS503P

Submitted to:
Dr. Paramveer Sandhu

1 Higher-order Goal

Campus placement preparation requires students to search through numerous company brochures, placement notices, interview experiences, eligibility criteria, aptitude resources, resume guidelines, and technical preparation material. Most of this information exists as unstructured PDF documents distributed across different platforms, making it difficult to retrieve relevant information quickly.

The primary objective of this project is to develop an intelligent AI-powered Placement Assistant that enables students to interact with placement-related documents using natural language. Instead of manually searching through hundreds of pages, students can simply ask questions and receive context-aware responses generated using Retrieval-Augmented Generation (RAG).

Beyond serving as a chatbot, the proposed system aims to become a centralized knowledge platform that assists students throughout the placement process while demonstrating practical applications of modern Artificial Intelligence technologies.

2 Time-to-value

The project will follow an incremental engineering approach where useful functionality is delivered early.

- **Iteration 1:** Develop a basic RAG chatbot capable of answering questions from uploaded placement documents.
- **Iteration 2:** Implement semantic document retrieval using embeddings and a vector database.

- **Iteration 3:** Add Resume Analysis functionality to evaluate resumes and provide AI-generated improvement suggestions.
- **Iteration 4:** Develop Company Eligibility Checker and Placement Dashboard.
- **Iteration 5:** Optimize performance, improve retrieval quality, conduct testing, and deploy the complete application.

The incremental approach allows meaningful software to be demonstrated early while continuously improving system quality throughout development.

3 Problem Statement

Students preparing for campus placements often face significant challenges in accessing accurate and relevant placement-related information. Resources such as company eligibility criteria, placement brochures, previous interview experiences, aptitude preparation material, resume guidelines, and technical preparation notes are typically scattered across multiple PDF documents, emails, and online portals.

The major challenges include:

- **Fragmented Information:** Placement resources are distributed across multiple sources, making retrieval time-consuming.
- **Manual Searching:** Students must manually browse lengthy documents to locate specific information.
- **Repeated Queries:** Placement coordinators repeatedly answer similar questions regarding eligibility, interview process, and company requirements.
- **Lack of Personalized Guidance:** Students often struggle to identify missing technical skills or determine whether they satisfy company eligibility requirements.
- **Resume Quality Issues:** Many resumes contain formatting mistakes, missing keywords, and weak project descriptions that reduce interview opportunities.

With increasing campus recruitment activities and growing volumes of placement documentation, manually managing this information becomes inefficient. An intelligent retrieval system capable of understanding natural language queries can significantly improve students' preparation experience while reducing repetitive administrative effort.

4 Proposed Solution

4.1 Overview

The proposed solution is a web-based AI Placement Assistant powered by Retrieval-Augmented Generation (RAG). The system combines semantic search with Large Language Models (LLMs) to provide accurate, context-aware responses grounded in institutional placement documents.

Instead of relying solely on the knowledge of an LLM, the system first retrieves the most relevant document sections from a vector database and then uses those retrieved passages as

context for answer generation. This significantly reduces hallucinations while improving factual accuracy.

The system will support multiple placement-related services through a unified interface.

4.2 Core Functionalities

1. Upload placement-related PDF documents.
2. Automatically extract and preprocess document text.
3. Divide documents into semantic chunks.
4. Generate vector embeddings using a pre-trained embedding model.
5. Store embeddings in a vector database (ChromaDB).
6. Retrieve the most relevant document chunks using semantic similarity search.
7. Generate grounded answers using the Gemini Large Language Model.
8. Analyze student resumes and provide improvement suggestions.
9. Check company eligibility based on student profile information.
10. Provide interview preparation guidance using previous interview experiences and technical resources.

4.3 Expected Benefits

- Faster retrieval of placement-related information.
- Reduced dependence on manual document searching.
- Accurate responses supported by institutional documents.
- Personalized preparation guidance.
- Improved resume quality.
- Better placement readiness for students.

4.4 System Workflow

The proposed workflow is as follows:

1. Placement documents are uploaded by the administrator.
2. The system extracts text from PDF files.
3. Documents are divided into smaller semantic chunks.
4. Vector embeddings are generated for each chunk.
5. Embeddings are stored inside ChromaDB.

6. A student submits a natural language query.
7. The query embedding is generated.
8. The retriever finds the most relevant document chunks.
9. Retrieved context is passed to the Gemini API.
10. The Large Language Model generates an answer grounded in retrieved information.

5 Solution Approach

The proposed system follows a modular architecture consisting of document processing, semantic retrieval, Large Language Model (LLM) inference, and web application layers. Rather than allowing the LLM to answer directly from its pretrained knowledge, the system retrieves relevant information from placement documents before generating responses. This Retrieval-Augmented Generation (RAG) pipeline ensures that answers remain grounded in verified institutional resources.

5.1 Document Processing Pipeline

The knowledge base consists of placement-related PDF documents including company brochures, eligibility criteria, interview experiences, resume guidelines, aptitude preparation material, and technical interview notes.

The preprocessing pipeline performs the following operations:

1. Upload PDF documents.
2. Extract text using PDF parsers.
3. Clean and normalize extracted text.
4. Split documents into overlapping semantic chunks.
5. Generate dense vector embeddings for each chunk.
6. Store embeddings together with metadata inside ChromaDB.

This preprocessing is performed only once for every uploaded document, allowing efficient retrieval during user interactions.

5.2 Retrieval-Augmented Generation Pipeline

When a student submits a query, the system executes the following sequence:

1. Convert the user query into an embedding.
2. Perform semantic similarity search within the vector database.
3. Retrieve the Top-K most relevant document chunks.
4. Construct an augmented prompt using the retrieved context.

5. Send the prompt to the Gemini Large Language Model.

6. Generate a grounded response based only on the retrieved information.

7. Display the answer together with the supporting document references.

This architecture significantly reduces hallucinations and improves factual consistency compared to conventional chatbot systems.

5.3 Resume Analysis Module

The Resume Analysis module allows students to upload resumes in PDF format.

The system performs:

- Resume parsing.
- Skill extraction.
- Keyword matching.
- Project evaluation.
- Technical skill recommendations.
- Resume improvement suggestions using Gemini.

This module assists students in improving resume quality before placement applications.

5.4 Company Eligibility Checker

Students enter their academic profile including:

- CGPA
- Branch
- Current semester
- Number of backlogs
- Technical skills

The system compares these details with company eligibility criteria extracted from placement documents and determines whether the student satisfies the requirements.

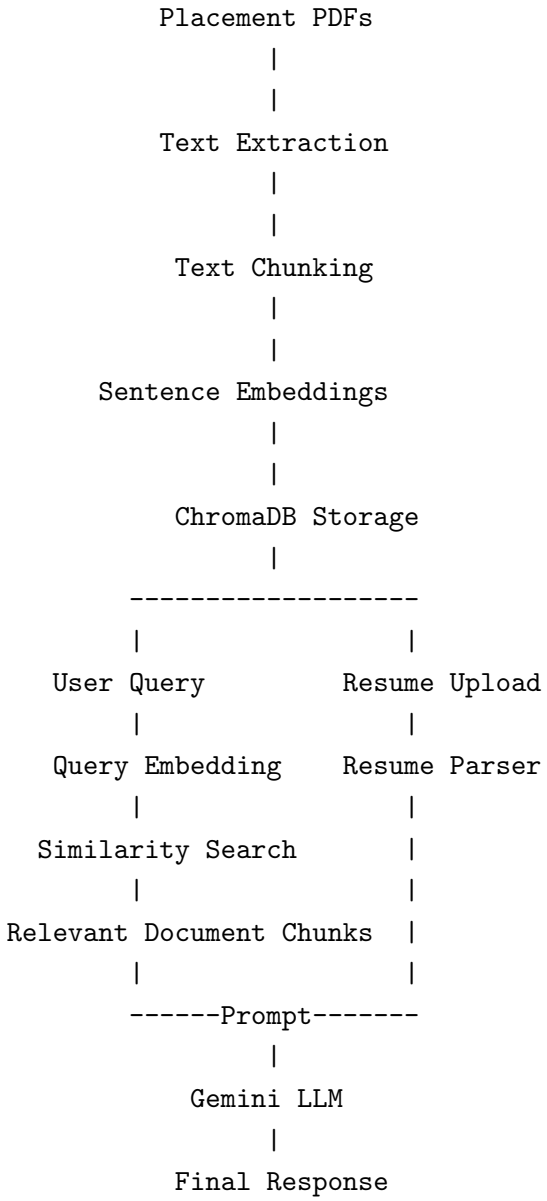
5.5 Technology Stack

The proposed software stack is shown below.

Frontend	Streamlit
Backend	FastAPI
Programming Language	Python
LLM	Google Gemini API
Embeddings	Sentence Transformers
Vector Database	ChromaDB
Framework	LangChain
Database	SQLite / PostgreSQL
Version Control	Git and GitHub
Deployment	Render / Railway

5.6 System Architecture

The system follows a layered architecture.



```
          |
Streamlit Interface
```

5.7 Software Engineering Considerations

The project follows modular software engineering principles.

- Separation of frontend, backend, retrieval, and AI modules.
- Reusable service-oriented architecture.
- RESTful API communication.
- Version control using Git.
- Modular code organization.
- Error handling and logging.
- Scalable vector database design.
- Easy integration of additional placement documents.

6 Evaluation Criteria

The success of the proposed AI Placement Assistant will be evaluated using measurable software engineering and Artificial Intelligence metrics.

6.1 Primary Evaluation Metrics

- **Retrieval Accuracy**

  Measure whether the retrieved document chunks are relevant to the user's query.

  Target:

  - Top-3 retrieval accuracy greater than 90%.

- **Response Relevance**

  Evaluate whether the generated answer correctly addresses the user's question using retrieved context.

  Target:

  - Average user rating above 4/5.

- **Response Time**

  Measure the end-to-end latency from query submission to answer generation.

  Target:

  - Less than 3 seconds for normal queries.

6.2 Secondary Evaluation Metrics

- Resume analysis accuracy.
- Company eligibility prediction correctness.
- Number of successfully processed documents.
- Number of supported concurrent users.
- API reliability.
- User satisfaction survey.
- Percentage reduction in manual searching time.

6.3 Testing Strategy

The system will be validated using multiple testing approaches.

- Unit Testing

  Each module including document parser, embedding generator, retriever, and backend API will be tested independently.

- Integration Testing

  Validate communication between frontend, backend, vector database, and Gemini API.

- Functional Testing

  Verify that all major user functionalities work correctly.

- Performance Testing

  Evaluate response latency under multiple simultaneous requests.

- User Acceptance Testing

  Students will interact with the application and provide feedback regarding usability and response quality.

7 Scalability

The proposed architecture has been designed to accommodate future expansion without major architectural modifications.

7.1 Scalable Knowledge Base

New placement documents can be uploaded at any time.

The embedding pipeline automatically indexes the newly uploaded documents without affecting existing knowledge.

7.2 Modular Architecture

The frontend, backend, retrieval engine, and AI components remain independent modules.

This allows future upgrades with minimal changes.

7.3 Database Scalability

Although ChromaDB will initially store embeddings locally, the system can later migrate to production-grade vector databases such as Pinecone, Weaviate, or Milvus without changing the application logic.

7.4 Deployment Scalability

The application can be deployed using Docker containers and hosted on cloud platforms including Render, Railway, AWS, or Microsoft Azure.

8 Software Engineering Practices

The project follows modern software engineering principles.

- Modular project structure.
- Object-oriented programming.
- REST API architecture.
- Git-based version control.
- Continuous Integration using GitHub Actions.
- Automated testing.
- Documentation using Markdown.
- Exception handling and logging.
- Configuration management using environment variables.

9 Project Deliverables

9.1 Phase 1

- Document upload functionality.
- PDF preprocessing.
- Semantic chunk generation.
- ChromaDB integration.
- Basic RAG chatbot.

9.2 Phase 2

- Resume Analysis module.
- Company Eligibility Checker.
- Placement dashboard.
- Student profile management.

9.3 Phase 3

- Admin panel.
- Deployment.
- Performance optimization.
- User authentication.
- Complete project documentation.

10 CI/CD and Deployment

To ensure maintainability and continuous development, the project will use a basic CI/CD pipeline.

- Source code hosted on GitHub.
- Automated testing using GitHub Actions.
- Automatic deployment after successful builds.
- Docker containerization.
- Cloud deployment on Render or Railway.

11 Risks and Mitigation

Although the proposed system is technically feasible, several challenges may arise during development and deployment. Appropriate mitigation strategies will be adopted to minimize these risks.

11.1 Potential Risks

- **Hallucination by Large Language Models**

  Even with Retrieval-Augmented Generation, the language model may occasionally generate responses that are not completely supported by the retrieved documents.

  Mitigation: Only the retrieved document chunks will be provided as context to the LLM. Source references will also be displayed alongside generated responses.

- **Poor Quality Documents**

  Scanned or poorly formatted PDF documents may reduce retrieval accuracy.

  Mitigation: Document preprocessing, text cleaning, and OCR techniques will be applied whenever necessary.

- **Embedding Quality**

  Semantic retrieval depends on the quality of document embeddings.

  Mitigation: Well-established Sentence Transformer embedding models will be used, and chunk sizes will be experimentally optimized.

- **API Availability**

  Cloud-based LLM APIs may become unavailable or exceed usage quotas.

  Mitigation: API errors will be handled gracefully, and cached retrieval results will be maintained wherever possible.

- **Scalability**

  As the number of placement documents increases, retrieval latency may increase.

  Mitigation: Efficient vector indexing techniques and scalable vector databases can be adopted in future deployments.

12 Future Scope

Although the initial implementation focuses on campus placements, the proposed architecture can be extended significantly.

Future improvements include:

- Voice-based interaction using Speech-to-Text.
- Multilingual support for placement documents.
- OCR support for scanned PDFs.
- Hybrid Retrieval (BM25 + Dense Retrieval).
- Cross-Encoder Re-ranking for improved retrieval quality.
- Personalized preparation roadmaps.
- Interview scheduling assistant.
- AI-generated mock interviews.
- Integration with university placement portals.
- Mobile application support.
- Analytics dashboard for placement officers.
- Agentic AI workflows for automated placement assistance.

13 Expected Outcomes

Upon successful completion of the project, the following outcomes are expected:

- A fully functional AI-powered Placement Assistant.
- Accurate retrieval of placement-related information using Retrieval-Augmented Generation.
- Intelligent resume analysis with AI-generated suggestions.
- Automated company eligibility verification.
- Reduced time required to search placement resources.
- Improved placement preparation experience for students.
- Demonstration of modern AI Engineering concepts including vector databases, embeddings, semantic search, and Large Language Models.

14 Conclusion

This project proposes the development of an Intelligent Placement Assistant that leverages Retrieval-Augmented Generation (RAG) to improve the accessibility and usability of campus placement resources.

Unlike conventional chatbots, the proposed system generates responses grounded in verified institutional documents through semantic retrieval, thereby improving reliability and reducing hallucinations. The solution integrates modern AI technologies including vector databases, embedding models, Large Language Models, and scalable web services within a modular software engineering architecture.

Beyond serving as an academic project, the proposed system has the potential to become a practical deployment for educational institutions by simplifying placement preparation, reducing repetitive administrative effort, and providing students with personalized guidance throughout the recruitment process.

References

[1] LangChain Documentation. Available: `https://python.langchain.com`

[2] ChromaDB Documentation. Available: `https://docs.trychroma.com`

[3] Google AI Studio. Available: `https://ai.google.dev`

[4] Reimers, N., and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT Networks," EMNLP, 2019.

[5] Lewis, Patrick et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.